112006265,
TING-YU TSAI,
EECS (GS)

Matrix Sum Report

1. Why is the program so slow?

Original code

```
  GNU nano 7.2
#include <iostream>
#include <ctime>
#include <chrono>

#define N 10000000
#define M (64 / sizeof(int)) // Cache line size: `getconf LEVEL1_DCACHE_LINESIZE`

int mat[N][M];

int main() {
  for (int i = 0; i < N; i++) {
    for (int j = 0; j < M; j++) {
      mat[i][j] = 1;
    }
  }

  int sum = 0;

  const auto t1 = std::chrono::high_resolution_clock::now();
  for (int i = 0; i < M; i++) {
    for (int j = 0; j < N; j++) {
      sum += mat[j][i];
    }
  }
  const auto t2 = std::chrono::high_resolution_clock::now();
  const auto int_ms = std::chrono::duration_cast<std::chrono::milliseconds>(t2 - t1);

  std::cout << "took: " << int_ms.count() << " ms" << std::endl;
  std::cout << "sum: " << sum << std::endl;
}
```

The program is slow because its memory access pattern has poor spatial locality. A cache line is typically 64 bytes and arrays benefit from spatial locality. If the program accesses matrix elements in an order that does not reuse nearby data well, cache misses increase, and each cache miss can stall the CPU. Therefore, changing the traversal order to access nearby elements consecutively improves cache utilization and performance.

The sum loop does:

```
for (int i = 0; i < M; i++) {
   for (int j = 0; j < N; j++) {
      sum += mat[j][i];
   }
```

112006265,
TING-YU TSAI,
EECS (GS)

}

So the access order is:

mat[0][0], mat[1][0], mat[2][0], ...

This is **column-wise access** on a **row-major array**.

Step by step:

1. CPU loads the cache line containing mat[0][0].
2. That cache line actually contains the whole row: mat[0][0] ~ mat[0][15].
3. But the next access is mat[1][0], which is in a **different row**, so a **different cache line**.
4. So most of the previous cache line is wasted.
5. This repeats over and over.

So each time the CPU fetches **64 bytes**, it immediately uses only **4 bytes** and then jumps away. That gives poor cache usage, more cache misses, and more stalls. Every cache miss can cause a **pipeline stall**, and DRAM is much slower than cache .

2. Verify the idea using perf stat -d ./mat_sum

3. How to improve the performance?

112006265,
TING-YU TSAI,
EECS (GS)

Modified code

```
  GNU nano 7.2
#include <iostream>
#include <ctime>
#include <chrono>

#define N 10000000
#define M (64 / sizeof(int)) // Cache line size: `getconf LEVEL1_DCACHE_LINESIZE`

int mat[N][M];

int main() {
  for (int i = 0; i < N; i++) {
    for (int j = 0; j < M; j++) {
      mat[i][j] = 1;
    }
  }

  int sum = 0;

  const auto t1 = std::chrono::high_resolution_clock::now();
  for (int j = 0; j < N; j++) {
    for (int i = 0; i < M; i++) {
      sum += mat[j][i];
    }
  }
  const auto t2 = std::chrono::high_resolution_clock::now();
  const auto int_ms = std::chrono::duration_cast<std::chrono::milliseconds>(t2 - t1);

  std::cout << "took: " << int_ms.count() << " ms" << std::endl;
  std::cout << "sum: " << sum << std::endl;
}
```

Use **row-wise traversal** in the sum loop.

```
for (int i = 0; i < N; i++) {
   for (int j = 0; j < M; j++) {
      sum += mat[i][j];
   }
}
```

Why this is faster

A cache line is typically **64 bytes**, and performance improves when code has good **cache locality**, especially **spatial locality** for arrays. Every **cache miss** can cause a **pipeline stall**.

Step by step

Access order is:

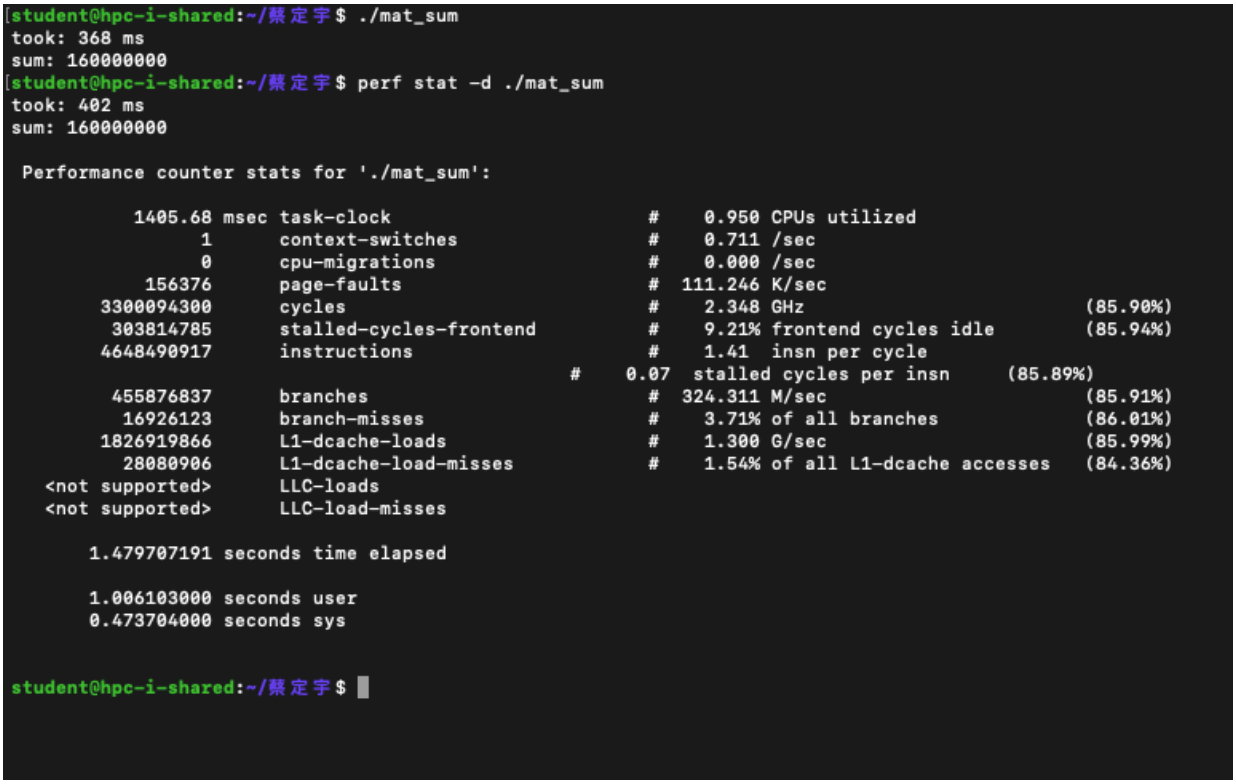
mat[0][0], mat[0][1], ..., mat[0][15]

1. CPU loads the cache line containing mat[0][0].

2. That cache line contains the whole row: mat[0][0] ~ mat[0][15].
3. The next access is mat[0][1], so it is still in the same cache line.
4. The CPU keeps using mat[0][2] ... mat[0][15] from that same cache line.
5. So one cache-line fetch is fully used.
6. Then CPU loads the next cache line for mat[1][0] ~ mat[1][15], moves to the next row and loads the next cache line.
7. This repeats row by row.
8. Therefore, cache usage is better, cache misses are fewer, and the program runs faster.

4. Verify that the modified program is faster

Modified version results



Time and perf comparison

Runtime comparison

- Original version: 830 ms

112006265,
TING-YU TSAI,
EECS (GS)

- Modified version: 368 ms
- Speedup
  - 830 / 368 ≈ 2.26
  - So the modified version is about 2.26× faster

Perf comparison

- Original
  - L1-dcache-load-misses: 152,522,772
  - L1 miss rate: 8.47%
- Modified
  - L1-dcache-load-misses: 28,080,906
  - L1 miss rate: 1.54%